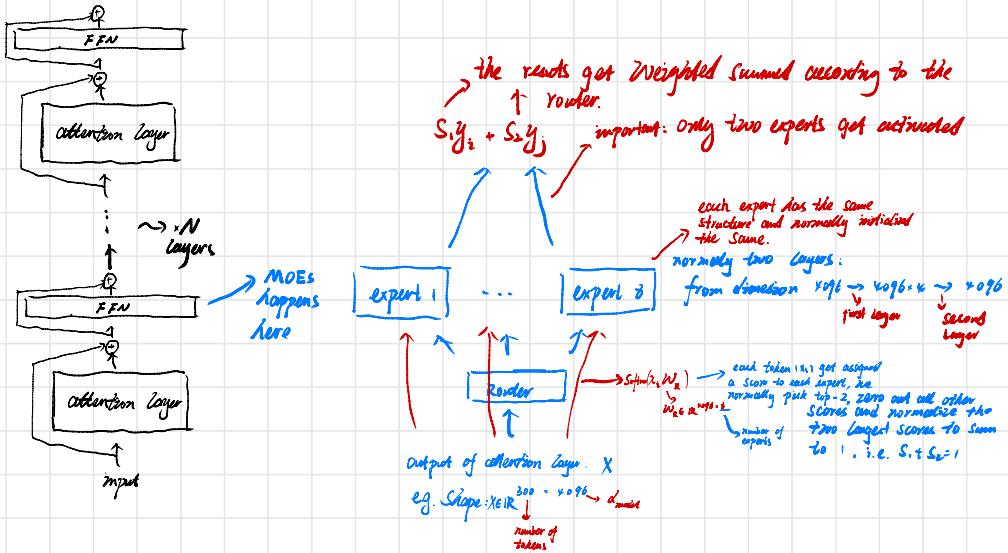


MOEs (Mixture of experts)



Notes:

1. how the routers get updated using gradient.

Introduce a constant mask matrix

$$\text{Softmax}(x_i W_R M_{\text{mask}})$$

→ update this

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class ExpertFFN(nn.Module):
    """A standard Feed-Forward Network acting as an individual expert."""
    def __init__(self, d_model, d_hidden):
        super().__init__()
        self.fc1 = nn.Linear(d_model, d_hidden)
        self.fc2 = nn.Linear(d_hidden, d_model)
        self.act = nn.GELU()

    def forward(self, x):
        # Example: 4096 -> 16384 -> 4096
        return self.fc2(self.act(self.fc1(x)))

class SparseMoE(nn.Module):
    def __init__(self, d_model, num_experts, top_k, d_hidden):
        super().__init__()
        self.num_experts = num_experts
        self.top_k = top_k

        # The Router Matrix (W_R)
        self.router = nn.Linear(d_model, num_experts, bias=False)

        # The N Experts
        self.experts = nn.ModuleList([
            ExpertFFN(d_model, d_hidden) for _ in range(num_experts)
        ])

    def forward(self, x):
        batch_size, seq_len, d_model = x.shape
        # Flatten batch and sequence dimensions: N = batch_size * seq_len
        x_flat = x.view(-1, d_model)

        # 1. Calculate router logits for the entire flattened batch
        # Shape: (N, num_experts)
        router_logits = self.router(x_flat)

        # 2. Dynamic Masking / Top-K Selection
        # Instead of literally adding negative infinity to a mask, PyTorch's topk
        # allows us to just extract the highest scores and do softmax only on those.
        # This is mathematically identical but much more computationally efficient.
        topk_logits, topk_indices = torch.topk(router_logits, self.top_k, dim=-1)

        # Softmax over only the Top-K choices so they sum to 1
        # Shape: (N, top_k)
        topk_weights = F.softmax(topk_logits, dim=-1)

        # 3. Dispatching to Experts
        # We prepare an empty tensor to accumulate the final weighted outputs
        final_output = torch.zeros_like(x_flat)
        # Iterate over the top_k choices (e.g., first choice, then second choice)
        for i in range(self.top_k):
            # Grab the expert indices and weights for this specific "choice" slot
            expert_indices = topk_indices[:, i] # Shape: (N,)
            weights = topk_weights[:, i].unsqueeze(1) # Shape: (N, 1)

            # Process tokens for each expert one by one
            for expert_idx in range(self.num_experts):
                # Create a boolean mask identifying which tokens go to THIS expert
                # This is the practical application of the dynamic mask we discussed:
                token_mask = (expert_indices == expert_idx)

                # If no tokens are routed to this expert in this batch, skip it to save compute
                if not token_mask.any():
                    continue

                # Extract the specific tokens meant for this expert
                expert_input = x_flat[token_mask]

                # Pass them through the expert
                expert_output = self.experts[expert_idx](expert_input)

                # Multiply the output by the routing probability (S, 1 * y, 1)
                weighted_output = expert_output * weights[token_mask]

                # Add the result to the final output tensor
                final_output[token_mask] += weighted_output

        # Reshape back to (batch_size, seq_len, d_model)
        return final_output.view(batch_size, seq_len, d_model)

```